Lab 5

Artificial Intelligence Lab

1 2 3 4

Program 1 **BFS Traversal & Tree on a Given Graph**

Perform BFS on the given graph starting from Node 0. Print the order in which nodes are visited. Also show the BFS tree by printing each node and its parent.

AI_lab_05 > bfs.py > bfs_traversal

```python
graph = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5, 6],
    3: [1, 4],
    4: [1, 3],
    5: [2],
    6: [2]
}
from collections import deque

def bfs_traversal(graph, start):
    visited = set()
    queue = deque([start])
    parent = {start: None}

    visited.add(start)

    print("BFS Order:", end=" ")

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = node
                queue.append(neighbor)

    print("\n\nBFS Tree (child : parent):")
    for node in parent:
        print(f"{node} : {parent[node]}")

bfs_traversal(graph, 0)
```

OUTPUT:——

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Us
BFS Order: 0 1 2 3 4 5 6

BFS Tree (child : parent):
0 : None
1 : 0
2 : 0
3 : 1
4 : 1
5 : 2
6 : 2
```

Program 2 **Find a Path Between Two Nodes**

Using BFS on the given graph, find if a path exists from Node 0 to Node 6. Print the complete path if found (e.g. 0 → 2 → 6), or print 'No path found'.

AI_lab_05 > path.py > bfs_path

```python
graph = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5, 6],
    3: [1, 4],
    4: [1, 3],
    5: [2],
    6: [2]
}
from collections import deque

def bfs_path(graph, start, end):
    visited = set()
    queue = deque([start])
    parent = {start: None}

    visited.add(start)

    while queue:
        node = queue.popleft()

        if node == end:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = node
                queue.append(neighbor)

    return None

path = bfs_path(graph, 0, 6)

if path:
    print("Shortest Path:", " → ".join(map(str, path)))
else:
    print("No path found")
```

OUTPUT:-

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -
Shortest Path: 0 → 2 → 6
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 3

Detect a Cycle in the Graph

Check whether the given graph contains a cycle using BFS. Print 'Cycle detected' or 'No cycle found'. Also test your program on a second graph (a simple tree) to verify the 'No cycle' case.

AI_lab_05 > cycle.py > detect_cycle

```python
graph = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5, 6],
    3: [1, 4],
    4: [1, 3],
    5: [2],
    6: [2]
}
from collections import deque

def detect_cycle(graph):
    visited = set()

    for start in graph:
        if start not in visited:
            queue = deque([(start, -1)])
            visited.add(start)

            while queue:
                node, parent = queue.popleft()

                for neighbor in graph[node]:
                    if neighbor not in visited:
                        visited.add(neighbor)
                        queue.append((neighbor, node))
                    elif neighbor != parent:
                        return True
    return False

if detect_cycle(graph):
    print("Cycle detected")
else:
    print("No cycle found")
```

OUTPUT:—

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python
Cycle detected
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 4 **Connected Components in the Graph**

Find all connected components in the graph using BFS. Print the nodes in each component separately.

AI_lab_05 > components.py > ...

```python
graph = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5, 6],
    3: [1, 4],
    4: [1, 3],
    5: [2],
    6: [2]
}
from collections import deque

def connected_components(graph):
    visited = set()

    for node in graph:
        if node not in visited:
            queue = deque([node])
            visited.add(node)
            component = []

            while queue:
                curr = queue.popleft()
                component.append(curr)

                for neighbor in graph[curr]:
                    if neighbor not in visited:
                        visited.add(neighbor)
                        queue.append(neighbor)

            print("Component:", component)

connected_components(graph)
```

OUTPUT:—

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % pyth
Component: [0, 1, 2, 3, 4, 5, 6]
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```